

Informe de Laboratorio: Pruebas sobre el comportamiento de la Memoria Caché

1st Justo Alfredo Perez Choque
dept. Ciencia de la Computación
Universidad Católica San Pablo
Arequipa, Perú
justo.perez@ucsp.edu.pe

I. INTRODUCCIÓN

El objetivo de este laboratorio es estudiar el comportamiento de la memoria caché. Se implementaron dos ejercicios, primero una comparación de dos variantes de bucles anidados en C++ y en GO, y se compararon sus tiempos de ejecución para observar cómo las diferencias en el acceso a la memoria pueden influir en el desempeño. Luego, se analizó la eficiencia de dos algoritmos de multiplicación de matrices, uno clásico y otro basado en bloques, evaluando su rendimiento en términos de *cache hits*, *cache misses*, y la gestión de la localidad espacial y temporal. El repositorio de todo el trabajo se encuentra en el repositorio[1].

II. IMPLEMENTACIÓN

A. Código del libro: Bucles anidados

El código utilizado para este experimento consiste en dos funciones (*test1* y *test2*) que realizan operaciones similares sobre una matriz *A* y dos vectores *x* e *y*.

La principal diferencia entre *test1* y *test2* radica en el orden de los bucles anidados:

- En *test1*, el bucle externo itera sobre el índice *i*, mientras que el bucle interno lo hace sobre el índice *j*.
- En *test2*, el orden de los bucles se invierte; el bucle externo itera sobre *j* y el interno sobre *i*.

Ambas funciones realizan la operación:

$$y[i] += A[i][j] \times x[j]$$

```
void test1(double** A, double* x, double* y, int MAX)
{
    for (int i = 0; i < MAX; i++)
        for (int j = 0; j < MAX; j++)
            y[i] += A[i][j] * x[j];
}

void test2(double** A, double* x, double* y, int MAX)
{
    for (int j = 0; j < MAX; j++)
        for (int i = 0; i < MAX; i++)
            y[i] += A[i][j] * x[j];
}
```

B. Código Multiplicación Matrices

Realicé dos algoritmos para la multiplicación de matrices: la multiplicación clásica utilizando tres bucles anidados y la multiplicación por bloques utilizando seis bucles anidados.

```
void multMatrizClasica(int **m1, int **m2, int **res,
    int N ) {

    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            for(int k = 0; k < N; k++){
                *(*(res+i)+j) += *(*(m1+i)+k) * *(*(
                    m2+k)+j);
            }
        }
    }
    return;
}

void multMatrizBloque(int **m1, int **m2, int **res,
    int N , int S){
    for (int ii = 0; ii < N; ii += S) {
        for (int jj = 0; jj < N; jj += S) {
            for (int kk = 0; kk < N; kk += S) {
                for (int i = ii; i < std::min(ii+S,N)
                    ; ++i) {
                    for (int j = jj; j < std::min(jj+
                        S, N); ++j) {
                        for (int k = kk; k < std:::
                            min(kk +S, N); ++k) {
                            res[i][j] += m1[i][k]*m2
                                [k][j];
                        }
                    }
                }
            }
        }
    }
}
```

III. RESULTADOS Y ANALISIS

A. Especificaciones del Procesador

Las pruebas de rendimiento fueron realizadas en un procesador **Intel Core i7-10700F**, cuyas especificaciones son las siguientes:

- **Arquitectura:** x86_64 (soporta **32-bit** y **64-bit**).
- **Número de núcleos e hilos:**
 - **Núcleos físicos:** 8
 - **Hilos de ejecución:** 16 (*Hyper-Threading* activado)
- **Frecuencias de operación:**
 - **Frecuencia base:** 2.90 GHz
 - **Frecuencia máxima (Turbo Boost):** 4.80 GHz
 - **Frecuencia mínima:** 800 MHz (modo de ahorro de energía)

- **Memoria caché:**
 - **Caché L1:**
 - * **Datos (L1d):** 256 KiB (8 instancias, 32 KiB por núcleo)
 - * **Instrucciones (L1i):** 256 KiB (8 instancias, 32 KiB por núcleo)
 - **Caché L2:** 2 MiB (8 instancias, 256 KiB por núcleo)
 - **Caché L3:** 16 MiB (compartida entre todos los núcleos)
- **Virtualización:** Soporte para VT-x (*Intel Virtualization Technology*).
- **Vulnerabilidades y mitigaciones:**
 - **Spectre v1 y v2:** Mitigado por microcódigo y software.
 - **Meltdown:** No afectado.
 - **L1TF, MDS, SRBDS, TAA:** No afectado o mitigado.
 - **Spec Store Bypass, Retbleed:** Mitigaciones activadas.

porque los elementos de una columna no están contiguamente almacenados en la memoria. Esto dificulta el aprovechamiento de la *localidad espacial*, resultando en una menor eficiencia en el uso de la caché.

B. Código del libro: Bucles anidados

La Tabla I y la Tabla II muestran que la primera forma tiene un mejor rendimiento que la segunda, especialmente a medida que el tamaño de la matriz aumenta. Esto se debe principalmente a cómo ambas formas aprovechan la memoria caché. La primera forma tiene un alto número de *cache hits* debido a que accede secuencialmente a la memoria por filas, lo que explota la *localidad espacial y temporal*. Esto significa que los datos adyacentes y recientemente utilizados están más frecuentemente disponibles en la caché, reduciendo la necesidad de acceder a la memoria principal.

Valor de MAX	Primera Forma (segundos)	Segunda Forma (segundos)
100	4.1969e-05	4.3164e-05
200	0.000173058	0.000200513
400	0.000657152	0.000898779
800	0.00266543	0.00427397
1600	0.010799	0.0286272
3200	0.0436007	0.134383
6400	0.176388	0.566714
12800	0.714516	3.0812
25600	3.40258	17.342

TABLE I: Tiempo de ejecución en segundos para las dos formas de bucles anidados en C++.

Valor de MAX	Primera Forma (segundos)	Segunda Forma (segundos)
100	0.007778	0.007468
200	0.026658	0.027440
400	0.106350	0.112743
800	0.428597	0.457661
1600	1.715543	1.908691
3200	6.905817	8.143203
6400	28.105395	32.631453
12800	112.481481	132.151404
25600	445.666284	532.672041

TABLE II: Tiempo de ejecución en segundos para las dos formas de bucles anidados en Go.

Por otro lado, la segunda forma accede a la matriz por columnas, lo que provoca un mayor número de *cache misses*

test1(double**, double*, double*, int)					
Types	Callers	All Callers	Callee Map	Source Code	
Event Type	Incl.	Self	Short	Formula	
Instruction Fetch	40.68	40.68	Ir		
L1 Instr. Fetch Miss	0.19	0.19	I1mr		
LL Instr. Fetch Miss	0.11	0.11	ILmr		
Data Read Access	39.99	39.99	Dr		
L1 Data Read Miss	16.64	16.64	D1mr		
LL Data Read Miss	11.65	11.65	DLmr		
Data Write Access	33.29	33.29	Dw		
L1 Data Write Miss	0.00	0.00	D1mw		
LL Data Write Miss	0.00	0.00	DLmw		
L1 Miss Sum	15.36	15.36	L1m = I1mr + D1mr + D1mw		
Last-level Miss Sum	10.44	10.44	LLm = ILmr + DLmr + DLmw		
Cycle Estimation	22.52	22.52	CEst = Ir + 10 L1m + 100 LLm		

(a) Análisis en KCacheGrind - C++ (Test 1)

test2(double**, double*, double*, int)					
Types	Callers	All Callers	Callee Map	Source Code	
Event Type	Incl.	Self	Short	Formula	
Instruction Fetch	40.68	40.68	Ir		
L1 Instr. Fetch Miss	0.28	0.28	I1mr		
LL Instr. Fetch Miss	0.22	0.22	ILmr		
Data Read Access	39.99	39.99	Dr		
L1 Data Read Miss	83.34	83.34	D1mr		
LL Data Read Miss	88.34	88.34	DLmr		
Data Write Access	33.29	33.29	Dw		
L1 Data Write Miss	0.00	0.00	D1mw		
LL Data Write Miss	0.00	0.00	DLmw		
L1 Miss Sum	76.92	76.92	L1m = I1mr + D1mr + D1mw		
Last-level Miss Sum	79.13	79.13	LLm = ILmr + DLmr + DLmw		
Cycle Estimation	64.07	64.07	CEst = Ir + 10 L1m + 100 LLm		

(b) Análisis en KCacheGrind - C++ (Test 2)

main.test1					
Types	Callers	All Callers	Callee Map	Source Code	
Event Type	Incl.	Self	Short	Formula	
Instruction Fetch	37.87	37.87	Ir		
Data Read Access	39.93	39.93	Dr		
Data Write Access	28.28	28.28	Dw		
L1 Instr. Fetch Miss	0.02	0.02	I1mr		
L1 Data Read Miss	14.24	14.24	D1mr		
L1 Data Write Miss	0.00	0.00	D1mw		
LL Instr. Fetch Miss	0.19	0.19	ILmr		
LL Data Read Miss	11.13	11.13	DLmr		
LL Data Write Miss	0.00	0.00	DLmw		
Conditional Branch	36.80	36.80	Bc		
Mispredicted Cond. Branch	2.02	2.02	Bcm		
Indirect Branch	0.00	0.00	Bi		
Mispredicted Ind. Branch	0.00	0.00	Bim		
L1 Miss Sum	13.20	13.20	L1m = I1mr + D1mr + D1mw		
Last-level Miss Sum	9.91	9.91	LLm = ILmr + DLmr + DLmw		
Mispredicted Branch	2.02	2.02	Bm = Bim + Bcm		
Cycle Estimation	22.59	22.59	CEst = Ir + 10 Bm + 10 L1m + 100 LLm		

(c) Análisis en KCacheGrind - Go (Test 1)

main.test2					
Types	Callers	All Callers	Callee Map	Source Code	
Event Type	Incl.	Self	Short	Formula	
Instruction Fetch	37.87	37.87	Ir		
Data Read Access	39.93	39.93	Dr		
Data Write Access	28.28	28.28	Dw		
L1 Instr. Fetch Miss	0.02	0.02	I1mr		
L1 Data Read Miss	85.56	85.56	D1mr		
L1 Data Write Miss	0.00	0.00	D1mw		
LL Instr. Fetch Miss	0.16	0.16	ILmr		
LL Data Read Miss	88.82	88.82	DLmr		
LL Data Write Miss	0.00	0.00	DLmw		
Conditional Branch	36.80	36.80	Bc		
Mispredicted Cond. Branch	2.02	2.02	Bcm		
Indirect Branch	0.00	0.00	Bi		
Mispredicted Ind. Branch	0.00	0.00	Bim		
L1 Miss Sum	79.29	79.29	L1m = I1mr + D1mr + D1mw		
Last-level Miss Sum	79.12	79.12	LLm = ILmr + DLmr + DLmw		
Mispredicted Branch	2.02	2.02	Bm = Bim + Bcm		
Cycle Estimation	60.77	60.77	CEst = Ir + 10 Bm + 10 L1m + 100 LLm		

(d) Análisis en KCacheGrind - Go (Test 2)

C. Multiplicación de Matrices

1) *Análisis y Complejidad:* La Tabla III muestra una comparación de los tiempos de ejecución para ambos enfoques de multiplicación de matrices.

Tamaño de Matriz	Tamaño de Bloque	Multiplicación Clásica (s)	Multiplicación por Bloques (s)
10	32	0.00131762	0.00272606
10	64	0.00131762	0.00105588
10	128	0.00131762	0.0010516
10	256	0.00131762	0.00105044
20	32	0.00379585	0.00784878
20	64	0.00379585	0.00784101
20	128	0.00379585	0.00781941
20	256	0.00379585	0.00780592
40	32	0.0304303	0.0613677
40	64	0.0304303	0.0630941
40	128	0.0304303	0.0614604
40	256	0.0304303	0.0615096
80	32	0.244707	0.47943
80	64	0.244707	0.477209
80	128	0.244707	0.484576
80	256	0.244707	0.486257
160	32	1.96934	3.79533
160	64	1.96934	3.79366
160	128	1.96934	3.77584
160	256	1.96934	3.86525
320	32	15.9093	30.6567
320	64	15.9093	30.1351
320	128	15.9093	30.7394
320	256	15.9093	30.7268
640	32	129.661	242.611
640	64	129.661	241.233
640	128	129.661	239.914
640	256	129.661	246.372
1280	32	1073.4	1949.68
1280	64	1073.4	2150.53
1280	128	1073.4	1908.87
1280	256	1073.4	1902.02

TABLE III: Tiempos de ejecución en segundos en C++.

Tamaño de Matriz	Tamaño de Bloque	Multiplicación Clásica (s)	Multiplicación por Bloques (s)
10	32	0.00395	0.00818
10	64	0.00395	0.00317
10	128	0.00395	0.00315
10	256	0.00395	0.00315
20	32	0.01139	0.02354
20	64	0.01139	0.02352
20	128	0.01139	0.02346
20	256	0.01139	0.02342
40	32	0.09129	0.18410
40	64	0.09129	0.18928
40	128	0.09129	0.18438
40	256	0.09129	0.18453
80	32	0.73412	1.43829
80	64	0.73412	1.43163
80	128	0.73412	1.45372
80	256	0.73412	1.45877
160	32	5.90802	11.38599
160	64	5.90802	11.38098
160	128	5.90802	11.32752
160	256	5.90802	11.59575
320	32	47.7279	91.9701
320	64	47.7279	90.4053
320	128	47.7279	92.2182
320	256	47.7279	92.1804
640	32	388.983	727.833
640	64	388.983	723.699
640	128	388.983	719.742
640	256	388.983	739.116
1280	32	3220.2	5849.04
1280	64	3220.2	6451.59
1280	128	3220.2	5726.61
1280	256	3220.2	5706.06

TABLE IV: Tiempos de ejecución en segundos en Go.

multMatrizBloque(int**, int**, int**, int, int)					
Types	Callers	All Callers	Callee Map	Source Code	
Event Type	Incl.	Self	Short	Formula	
Instruction Fetch	65.78	54.01	Ir		
Data Read Access	64.98	49.54	Dr		
Data Write Access	82.11	46.77	Dw		
L1 Instr. Fetch Miss	0.32	0.32	l1mr		
L1 Data Read Miss	22.54	22.54	D1mr		
L1 Data Write Miss	0.00	0.00	D1mw		
LL Instr. Fetch Miss	0.31	0.31	lLmr		
LL Data Read Miss	0.20	0.20	Dlmr		
LL Data Write Miss	0.00	0.00	DLmw		
Conditional Branch	72.52	36.26	Bc		
Mispredicted Cond. Branch	97.17	97.16	Bcm		
Indirect Branch	0.00	0.00	Bi		
Mispredicted Ind. Branch	0.00	0.00	Bim		
L1 Miss Sum	22.53	22.53	L1m = l1mr + D1mr + D1mw		
Last-level Miss Sum	0.20	0.20	LLm = lLmr + DLmr + DLmw		

(a) Análisis en KCacheGrind - C++ (Multiplicación clásica)

multMatrizClasica(int**, int**, int**, int)					
Types	Callers	All Callers	Callee Map	Source Code	
Event Type	Incl.	Self	Short	Formula	
Instruction Fetch	34.17	34.17	Ir		
Data Read Access	34.98	34.98	Dr		
Data Write Access	17.80	17.80	Dw		
L1 Instr. Fetch Miss	0.20	0.20	l1mr		
L1 Data Read Miss	77.46	77.46	D1mr		
L1 Data Write Miss	0.00	0.00	D1mw		
LL Instr. Fetch Miss	0.18	0.18	lLmr		
LL Data Read Miss	99.80	99.80	Dlmr		
LL Data Write Miss	0.00	0.00	DLmw		
Conditional Branch	27.41	27.41	Bc		
Mispredicted Cond. Branch	2.81	2.81	Bcm		
Indirect Branch	0.00	0.00	Bi		
Mispredicted Ind. Branch	0.00	0.00	Bim		
L1 Miss Sum	77.45	77.45	L1m = l1mr + D1mr + D1mw		
Last-level Miss Sum	99.29	99.29	LLm = lLmr + DLmr + DLmw		

(b) Análisis en KCacheGrind - C++ (Multiplicación por bloques)

La multiplicación de matrices clásica tiene una complejidad algorítmica de $O(N^3)$, donde N es el tamaño de la matriz. Este algoritmo itera a través de cada elemento de la matriz resultante y, para cada uno, realiza una suma de productos sobre la fila de la primera matriz y la columna de la segunda matriz. Debido a su acceso a memoria en un patrón no secuencial (acceso por columnas), este enfoque puede resultar en un alto número de *cache misses*, ya que los datos necesarios pueden no estar disponibles en la caché, obligando al algoritmo a acceder a la memoria principal con frecuencia.

En contraste, la multiplicación de matrices por bloques también tiene una complejidad algorítmica de $O(N^3)$, pero su rendimiento es optimizado al dividir las matrices en submatrices o bloques. Este enfoque mejora la localidad espacial y temporal, permitiendo que los datos accedidos recientemente se mantengan en la caché por más tiempo. Durante la ejecución, los bloques se cargan en la caché, lo que minimiza la cantidad de accesos a la memoria principal y reduce significativamente el número de *cache misses* en comparación con la multiplicación clásica.

Al ejecutar ambos algoritmos paso a paso, se puede observar que la multiplicación por bloques mejora el movimiento de datos entre la memoria principal y la caché. Esto se traduce en un rendimiento superior, especialmente en matrices de mayor tamaño, donde el acceso secuencial a los bloques reduce la latencia de memoria y optimiza el uso de la caché. En resumen, aunque ambos métodos tienen la misma complejidad algorítmica, la implementación por bloques ofrece una mejora considerable en la eficiencia práctica debido a un mejor manejo de la memoria caché.

2) Análisis con herramientas Valgrind y Kcachegrind:

La Figura 2 muestra un resumen de los resultados con una matriz de tamaño 640, obtenidos mediante las herramientas *Valgrind* y *Kcachegrind*. Se observa que la multiplicación clásica de matrices presenta un elevado número de *cache misses* en L1 para lectura de datos (**D1mr**), alcanzando un 77.46%. Además, los fallos en la última caché de datos (**DLmr**) llegan al 99.80%, lo que indica que los datos deben ser constantemente recuperados desde la memoria principal, penalizando el rendimiento del algoritmo.

En contraste, la versión optimizada por bloques mejora significativamente el aprovechamiento de la caché. Aunque la ejecución requiere un mayor número de accesos a memoria para escritura (**Dw**), la cantidad de fallos en la caché L1 es considerablemente menor, evidenciando una mejor explotación de la localidad de datos. Esto reduce la latencia causada por accesos repetidos a la memoria principal y mejora el desempeño general.

Por otro lado, la gestión de las escrituras en ambos algoritmos se mantiene eficiente, sin presentar *cache misses* en la caché de escritura (**D1mw**). Esto sugiere un adecuado uso de estrategias como *write-through* y *write-back*, minimizando la penalización por escrituras en memoria.

En términos de predicción de saltos, la multiplicación clásica exhibe un porcentaje elevado de ramas condicionales mal predichas (**Bcm**), lo que puede causar interrupciones en

Fig. 2: Comparación del uso de caché en C++ y Go con KCacheGrind

el flujo de ejecución y reducir la eficiencia del pipeline del procesador. En cambio, la versión por bloques mantiene un mejor control sobre la predicción de saltos, reduciendo la penalización por cambios inesperados en el flujo del programa.

A pesar de que la multiplicación por bloques ejecuta un número mayor de instrucciones, su mejor aprovechamiento de la jerarquía de memoria compensa este costo adicional, reduciendo los *cache misses* y optimizando el tiempo de ejecución.

IV. CONCLUSIONES

En el análisis del intercambio de índices para acceso a matrices, se concluye que la primera forma de acceder a los elementos por filas es significativamente más eficiente que la segunda forma, que accede por columnas. Esto se debe al mejor aprovechamiento de la localidad espacial y temporal, lo que resulta en un mayor número de *cache hits* y, por ende, en un menor tiempo de ejecución.

En cuanto a la multiplicación de matrices, el uso de un algoritmo de bloques demuestra una clara ventaja en términos de eficiencia en el manejo de la caché. A pesar de ejecutar más instrucciones, su capacidad para reducir los *cache misses* le permite superar a la multiplicación clásica, especialmente con matrices de gran tamaño. La optimización del uso de la memoria caché es, por lo tanto, un factor clave para mejorar el rendimiento en operaciones matriciales complejas.

REFERENCES

- [1] Justo Perez. Repositorio de github - trabajo comparación de cache. <https://github.com/just-0/Paralela>, 2025.